Audit doublons de règles

3 mai 2026 — réponse à ta question : "qu'est-ce que c'est cette histoire ?"

En une phrase

Les doublons de règles viennent **d'un défaut de conception** dans la fonction qui vérifie si une règle existe déjà avant d'en créer une nouvelle. Cette vérification est **trop stricte** : elle considère que `Comportement` et `comportement` sont deux catégories différentes, donc elle laisse passer les deux. Quand tu redéploies l'app sur Railway, ce défaut crée des doublons à chaque fois.

Comment ça se passe concrètement

1. Au démarrage de l'app

À chaque démarrage du serveur (donc à chaque redéploiement Railway, qui arrive à chaque fois que je `git push`), Raya exécute un bout de code qui dit :

> *"Si l'utilisateur n'a aucune règle initiale, je lui en crée."*

Cette vérification est **correcte** : si tu as déjà 100 règles, le code passe son chemin et ne réinjecte rien.

2. Mais il y a un endpoint admin qui force le re-seed

Dans le panneau super-admin, il y a un bouton qui dit "Re-seed ce tenant". Quand tu cliques dessus (ou quand un script appelle cet endpoint), il **force** la création des règles initiales **sans** vérifier si elles existent déjà.

Et c'est là que le problème démarre.

3. La fonction `save_rule` qui devrait dédupliquer

Quand on essaie de créer une règle, `save_rule` regarde dans la base si une règle "identique" existe déjà.

Sa définition de "identique" est :

- **même utilisateur**
- **même tenant**
- **même catégorie EXACTE** *(c'est ici le piège)*
- **même texte** (en ignorant les accents et les flèches)

Donc si une règle existe avec `category = "Comportement"` et que je veux insérer la même chose avec `category = "comportement"` (minuscule), la fonction dit *"je n'ai pas trouvé de doublon"* et **crée une deuxième règle**.

Résultat : pour une même idée, on se retrouve avec 2, 3, parfois 4 lignes en base.

Pourquoi cette nuit le job n'a pas fait le ménage

J'ai créé hier soir une nouvelle couche dans le job hebdomadaire qui répare ça : elle uniformise toutes les variantes de catégorie (`comportement` → `Comportement`, etc.) **avant** de chercher les doublons. Cette couche **a bien fonctionné** : 16 règles ont été recatégorisées proprement.

Mais ensuite, la couche qui devait fusionner les doublons textuels a échoué **silencieusement** à cause d'un autre bug que je n'avais pas vu : quand le code essaie de tracer la fusion dans la table d'historique, la base de données refuse l'écriture car le type de changement `merged_optimizer` n'était pas dans la liste autorisée. L'erreur a été interceptée sans planter le job, donc dans les logs on voyait simplement "0 fusion".

J'ai corrigé ce bug en base ce matin (j'ai ajouté `merged_optimizer` à la liste autorisée). Et j'ai relancé manuellement la fusion : **16 règles ont été fusionnées**, on est passé de 176 à 160 règles actives.

Mais ce n'est pas une vraie solution car :

1. Le bug n'est corrigé qu'en base — pas dans le code source. Si on redéploie sur une nouvelle instance, le bug revient.
2. Surtout, **la racine du problème reste : `save_rule` continue de créer des doublons casse à chaque re-seed.**

Pourquoi 16 nouvelles règles ce matin à 6h25

Quand j'ai poussé mes 8 commits hier soir vers minuit, Railway a redéployé l'app. Pendant le redémarrage, *quelque chose* a déclenché un re-seed (probablement l'un des endpoints admin appelé en boucle ou une logique de migration). 16 règles seed sont apparues, en minuscule, créant 16 nouveaux doublons avec les versions canoniques en majuscule qui existaient déjà.

C'est exactement la même mécanique qu'à chaque fois : tous les jours où tu vois `nb_seeds > 0` dans le tableau ci-dessous correspondent à des moments où l'app a redémarré ou où un re-seed a été déclenché.

Jour	Nouveaux seeds créés
23 avril	**28**
24 avril	12
9 avril	14
1er mai	**11**
27 avril	4
2 mai (hier)	4
3 mai (cette nuit)	**4**

À chaque pic, ce sont des doublons casse qui se créent.

Les 3 problèmes à régler ensemble

Problème 1 — La déduplication de `save_rule` est aveugle à la casse

Symptôme : `Comportement` et `comportement` passent comme 2 catégories.

Cause : la requête SQL utilise `WHERE category = %s` au lieu de comparer la version **normalisée** des deux catégories.

Solution : modifier la requête pour qu'elle compare des **signatures** de catégorie (lower + sans accent + séparateurs unifiés), comme on fait déjà pour le texte de la règle. Concrètement c'est ajouter une fonction

de normalisation à la requête, pas un appel IA.

> ⚠️ ***Important pour toi** : ce n'est PAS le code que j'avais mis hier qui faisait un SELECT par appel. C'est juste **ajouter une fonction SQL dans la requête existante** qui tourne déjà à chaque save_rule. Aucun coût supplémentaire (la requête tourne de toute façon).*

Problème 2 — Le job hebdomadaire échouait silencieusement

Symptôme : Layer A0 marche, Layer A trouve des candidats mais ne fusionne rien.

Cause : la table d'historique refuse les nouveaux types de changements (`merged_optimizer` , `recategorized_optimizer`).

Solution : ajouter ces types à la liste autorisée. Fait en base ce matin, à pousser dans le code (1 ligne dans `database_migrations.py`).

Problème 3 — Les seeds eux-mêmes utilisent des catégories minuscules incohérentes

Symptôme : dans `seeding.py` , les profils sont définis avec `category="comportement"` , `category="regroupement"` , `category="tri_mails"` (minuscule, underscore).

Cause : ces noms de catégorie ont été choisis comme **clés de dictionnaire Python** au début, sans réfléchir à la forme finale qu'aurait la catégorie en base.

Solution : harmoniser les seeds pour utiliser directement les formes canoniques (`Comportement` , `Regroupement` , `Tri mails` , `Mémoire`). Pas de logique en plus, juste renommer les clés dans le fichier seeding.py.

La solution que je propose

Une seule règle, un seul endroit, le dimanche soir. Conforme à ta vision.

Mais pour que cette règle fonctionne, il faut que **les briques de base** soient propres :

1. **Corriger le bug de la table d'historique** (Problème 2)
   - Pousser le fix dans `database_migrations.py`
   - 1 ligne, pas d'impact métier
2. **Harmoniser les seeds** (Problème 3)
   - Renommer 4 clés dans `seeding.py`
   - Aucun appel IA, aucun coût permanent
3. **Rendre `save_rule` insensible à la casse pour la catégorie** (Problème 1)
   - Modifier la requête SQL existante (pas en ajouter une nouvelle)
   - Aucun appel IA, aucun coût supplémentaire
   - C'est juste reconnaître que `Comportement` = `comportement` au moment de chercher si la règle existe déjà
   - **Ce n'est PAS le code coûteux que j'avais mis hier**
4. **Le job du dimanche reste en place**
   - Layer A0 (canonisation)
   - Layer A (fusion doublons)

- Layer B (contradictions)
- Layer C (oubli doux)
- Si malgré les 3 fixes ci-dessus il reste des doublons un dimanche, le job les nettoie

Avec ces 3 fixes au cœur, **on arrête de créer des doublons en amont**. Le job du dimanche n'a plus qu'à gérer les vrais doublons sémantiques (règles différentes mais qui veulent dire la même chose), pas les 16 doublons casse créés à chaque re-seed.

Coût comparé des 2 approches

Approche que tu as refusée hier (et tu avais raison)

À chaque création de règle, on faisait **un nouveau SELECT en base** pour chercher les variantes de catégorie. Une trentaine de règles créées par jour = 30 requêtes en plus. Pas catastrophique mais inutile.

Approche proposée maintenant

Pas de nouvelle requête. On **modifie la requête existante** de `save_rule` pour qu'elle compare la signature de la catégorie en plus de son texte. Le moteur SQL fait ça en quelques microsecondes, c'est gratuit.

	Approche refusée hier	Approche proposée
Nouvelle requête à chaque save_rule	Oui	Non
Appel IA supplémentaire	Non	Non
Coût/jour	~30 requêtes en plus	0
Empêche les doublons casse	Oui	Oui
Tient sur 7 jours sans intervention	Oui	Oui

Ma recommandation

On applique les 3 fixes ensemble (problèmes 1, 2, 3). Aucun n'augmente la facture IA. Tous trois sont **de la mécanique simple** dans du code Python/SQL. Et ils règlent la racine du problème.

Une fois ces 3 fixes en place, le job hebdo du dimanche fait son travail proprement, et tu n'as plus besoin d'y revenir.

Tu valides cette direction ?

Si oui, je code les 3 modifs en 1 commit propre, je pousse, et on regarde dimanche prochain ce qu'il reste à faire (idéalement : presque rien).

Si tu veux d'abord qu'on discute autre chose, je n'écris pas une ligne de code de plus.